

What is algorithm ?

Language independent, well defined and arranged procedure for solving a problem / performing computation.

Why algorithm ?

You have given a bunch of number and need to sort.

You have given road network and have to find shortest path from location A to B

You have given a bunch of task with deadline and need to find whether the task are computable or not within respective deadline.

Why Algorithm ?

1. Radar Communication(shortest path algorithm)
2. Cryptography(authentication, encryption)
3. Database(Query Optimization algorithms)
4. Data Mining(Classification algorithms)
5. Computer graphics(geometric algorithms)
6. Computational biology(DP algorithm)
7. Search Engine (page ranking algorithms)
8. Advertising, marketing, recommendation....

Algorithms Properties

Input(s)/output(s): There must be some inputs from the standard set of inputs and an algorithm's execution must produce outputs(s).

Definiteness: Each step must be clear and unambiguous.

Finiteness: Algorithms must terminate after finite time or steps.

Correctness: Correct set of output values must be produced from the each set of correct inputs.

Effectiveness: Each step must be carried out in finite time.

Here we deal with correctness and finiteness.

Expressing Algorithms

There are many ways of expressing algorithms; the order of ease of expression is natural language, pseudo code and real programming language syntax.

How Efficient is an exponential algorithm ?

If an algorithm needs 2^n steps for execution, then when $n=40$, the number of steps needed is approximately 1.1×10^{12} . On computer performing one billion (1×10^9) steps per second, this would require about 18.3 minutes.

If $n=50$, the same algorithm would run for about 13 days on this computer. When $n=60$, about 310.56 years are required to execute the algorithm and when $n=100$, about 4×10^{13} years are needed. So we may conclude that the utility of algorithms with exponential complexity is limited to small n (typically ≤ 40)

Random Access Machine Model

This RAM model is the base model for our study of design and analysis of algorithms to have design and analysis in machine independent scenario. In this model each basic operations (+, -) takes 1 step,

- loops and subroutines are not basic operations.
- Each memory reference is 1 step. We measure run time of algorithm by counting the steps.

Counting Execution Steps

	Statement	s/e	Frequency	Total steps
1	algorithm sum(a,n)	0	-	0
2	{	0	-	0
3	s=0;	1	1	1
4	for i=1 to n do	1	n+1	n+1
5	s=s+a[i];	1	n	n
6	return s;	1	1	1
7	}	0	-	0
	Total			2n+3

Counting Execution Steps

	Statement	s/e	Frequency	Total steps
1	algorithm sum(a,b,c,m,n)	0	-	0
2	{	0	-	0
3	for i=1 to m do	1	m+1	m+1
4	for j=1 to n do	1	m(n+1)	mn+m
5	c[i,j]=a[i,j]+b[i,j];	1	mn	mn
6	}	0	-	0
	Total			2mn+2m+1

Let's count executing steps

```
module {  
    int a;  
    a=1;  
    a ++;  
}
```

.....

$T(n) = O(1)$ how ?

Let's count executing steps

```
module
```

```
  for (i=1; i<=n; i++)
```

```
  {
```

```
    print(i);
```

```
  }
```

.....

$T(n) = O(n)$ how ?

Lets count executing steps

```
module
```

```
    i =0
```

```
While (i<n)
```

```
{    print(i);
```

```
    i++;
```

```
}
```

.....

$T(n) = O(n)$ how ?

Lets count executing steps

module

if (condition)

{

statements; $O(n^2)$

}

Else

statements; $O(n)$

.....

$T(n) = O(n^2)$

Lets count executing steps

module:

```
    for ( ; ; )    //  $O(n^2)$ 
    {
        for ( ; ; )
        {
            statements; //  $O(n)$ 
        }
    }
```

$T(n) = O(n^3)$

Let's Analyze using RAM model

```
1 module ()
2
3   { int a, b, c, count =0;    // O(1)
4
5       For(a=n/2;a<=n; i++) //O(n)
6
7       { For(b=1;b<=n;2*b)//O(logn)|
8
9           { For(c=1;c+n/2<=n; c++)//O(n)
10              { count++;
11                 printf("Message !")} // O(1)
12
13          }
14
15      }      // T(n) = n*logn*n= O(n^2.logn)
16 }
```

Let's Analyze using RAM model

```
1 module ()
2
3 { int a, b, c, count =0; // O(1)
4
5     For(a=n/2;a<=n; i++) //O(n)
6
7     { For(b=1;b<=n;2*b)//O(logn)
8
9         { For(c=1;c+n/2<=n; c++)//O(n)
10             { count++;
11               printf("Message !")} // O(1)
12         }
13     }
14
15     // T(n) = n*logn*n= O(n^2.logn)
16 }
```

First for loop

Iteration 1, $a = n/2$

Iteration 2, $a = n/2+1$

Iteration 3, $a = n/2+2$

.

.

.

Iteration k, $a = n/2 + k - 1 = n$

$n/2 + k - 1 = n$

$\Rightarrow k = n - n/2 + 1$

$= n/2 + 1$

Let's Analyze using RAM model

```
1 module ()
2
3 { int a, b, c, count =0; // O(1)
4
5     For(a=n/2;a<=n; i++) //O(n)
6
7     { For(b=1;b<=n;2*b)//O(logn)
8
9         { For(c=1;c+n/2<=n; c++)//O(n)
10             { count++;
11                 printf("Message !")} // O(1)
12             }
13
14     }
15
16 }
```

Second for loop

Iter 1 , $b = 1 = 2^0$

Iter 2 , $b = 2 = 2^1$

Iter 3 , $b = 4 = 2^2$

Iter 4, $b = 8 = 2^3$

...

Iter p , $b = 2^{p-1} = n$

$$n = 2^{p-1}$$

$$(p - 1)\log_2 = \log n$$

$$p = \log n / \log_2 + 1$$

Let's Analyze using RAM model

```
1 module ()
2
3 { int a, b, c, count =0; // O(1)
4
5     For(a=n/2;a<=n; i++) //O(n)
6
7     { For(b=1;b<=n;2*b)//O(logn)
8
9         { For(c=1;c+n/2<=n; c++)//O(n)
10             { count++;
11                 printf("Message !")} // O(1)
12             }
13
14     }
15
16 }
```

Third for loop

Iter 1 , c= 1

Iter 2 , c= 2

Iter 3 , c= 3

Iter 4, c = 4

...

Iter p ,c = p=n/2

=n/2

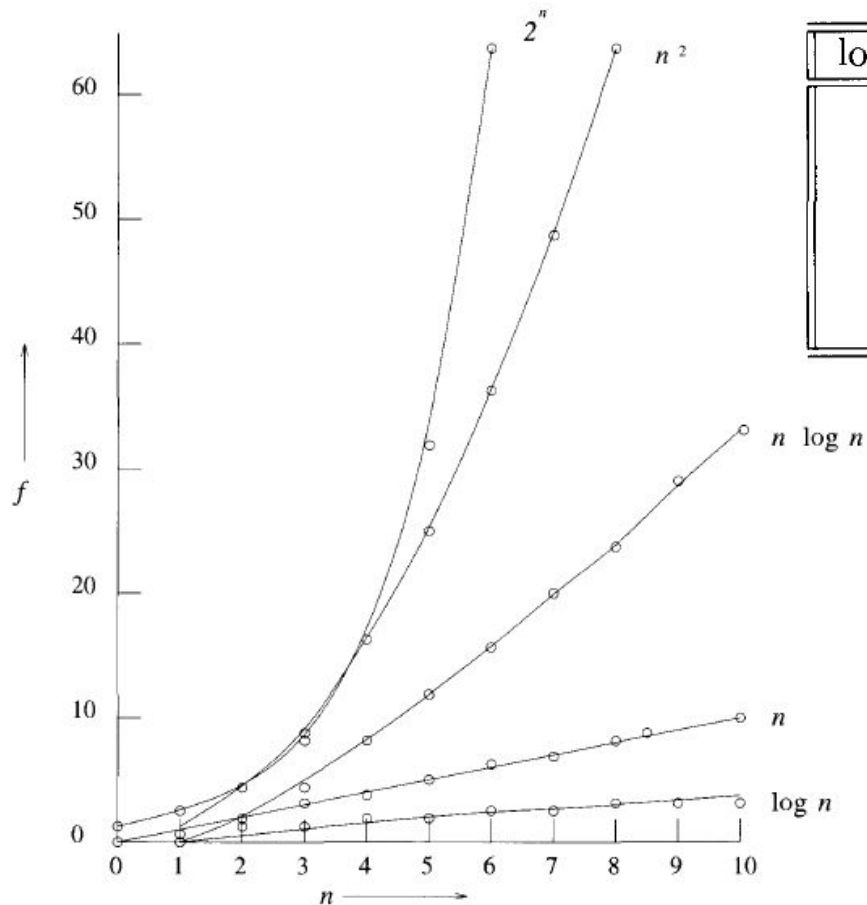
Factorial Finding

```
1.  #include<stdio.h>
2.  int main(){
3.      int x,fact=1,n;    // 1 step
4.      printf("Enter a number to find factorial: "); // 1 step
5.      scanf("%d",&n); // 1 step
6.      for(x=1; x<=n; x++) // n+1 steps
7.          fact=fact*x; // n step
8.      printf("Factorial of %d is: %d", n, fact); // 1 steps
9.      return 0; // 1 steps
10. }
```

Time Complexity, $T(n) = 1 + 1 + 1 + (n + 1) + n + 1 + 1 = O(n)$

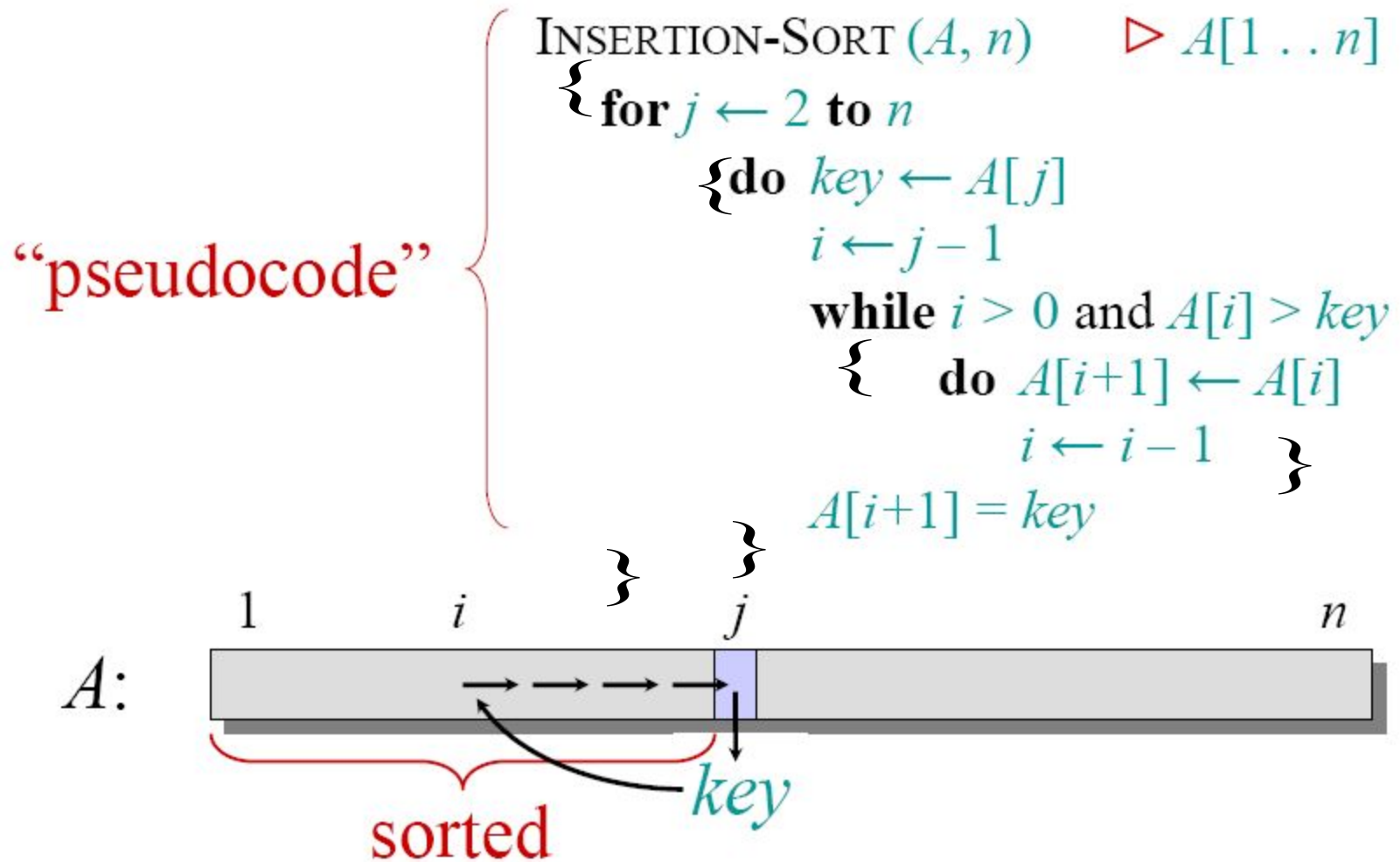
Space Complexity, $S(n) = 3 = O(1)$

How Computational time Increases with Increase of problem Size



$\log n$	n	$n \log n$	n^2	n^3	2^n
0	1	0	1	1	2
1	2	2	4	8	4
2	4	8	16	64	16
3	8	24	64	512	256
4	16	64	256	4,096	65,536
5	32	160	1,024	32,768	4,294,967,296

Insertion Sort



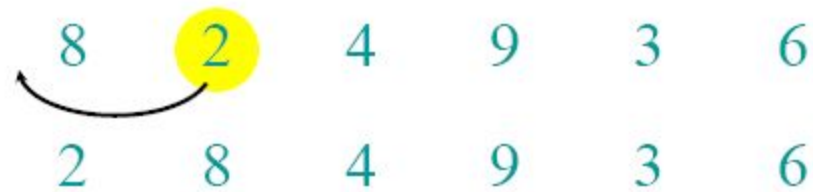
Insertion tracing

8 2 4 9 3 6

Insertion tracing



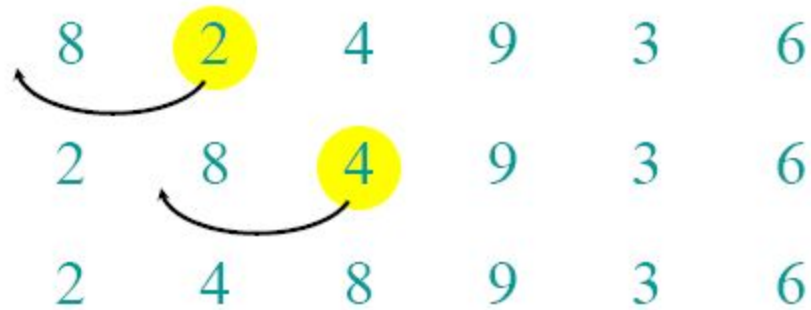
Insertion tracing



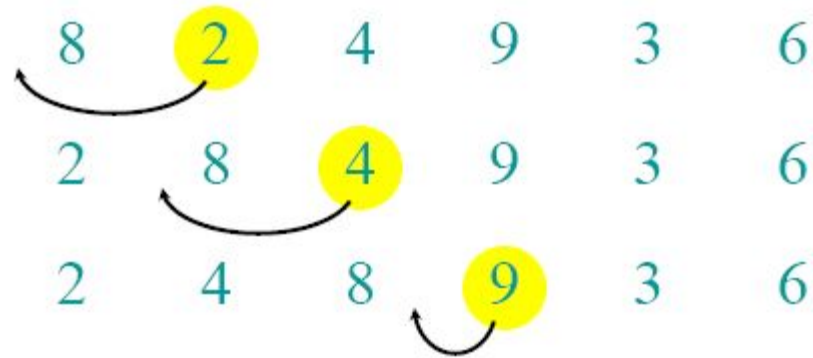
Insertion tracing



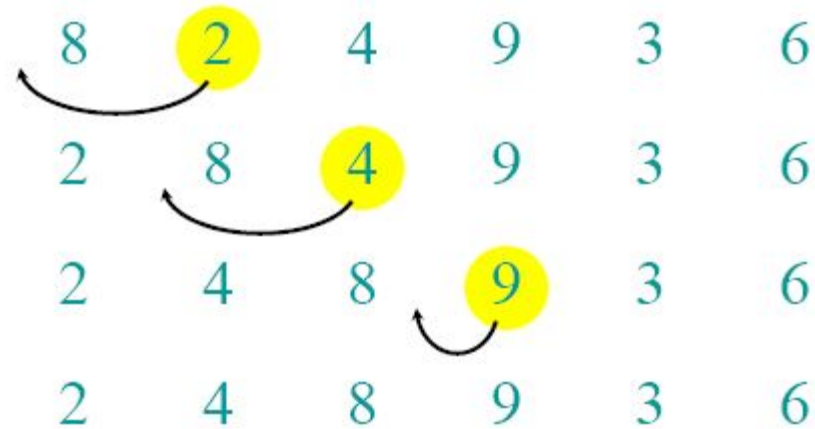
Insertion tracing



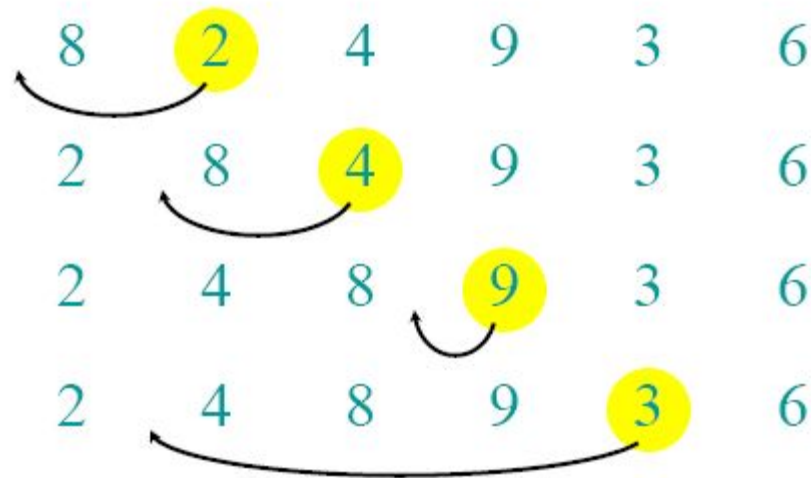
Insertion tracing



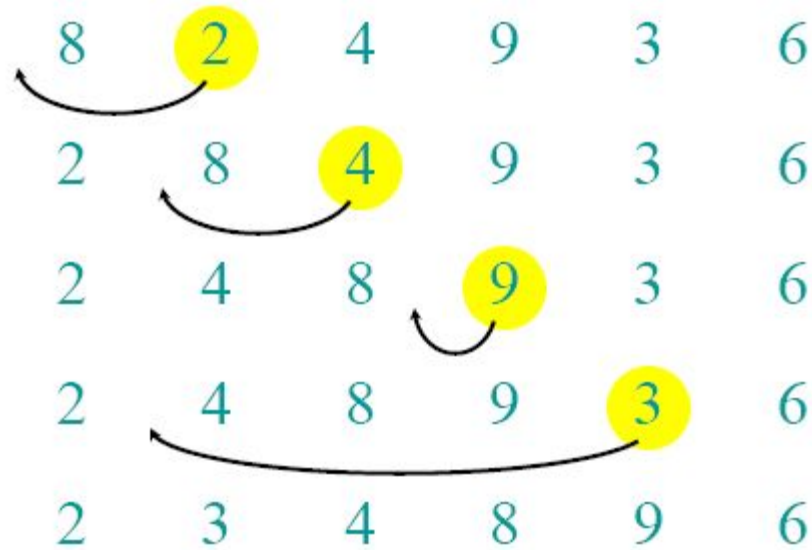
Insertion tracing



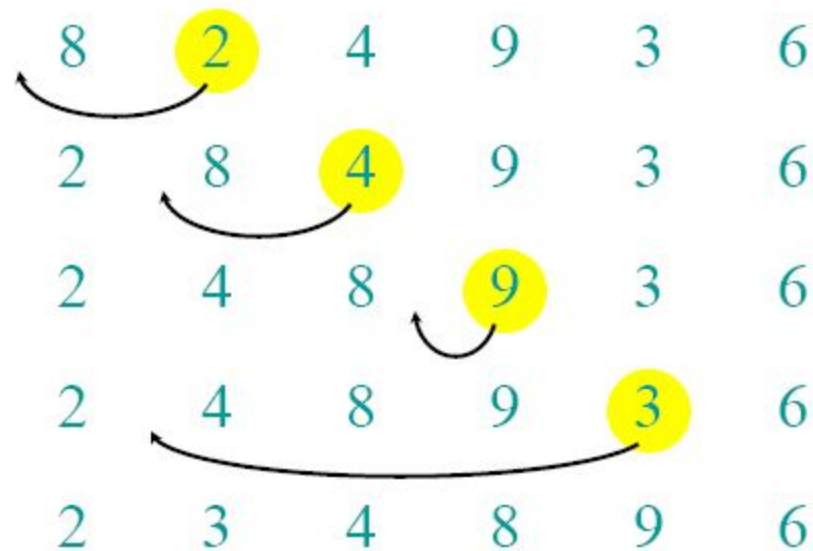
Insertion tracing



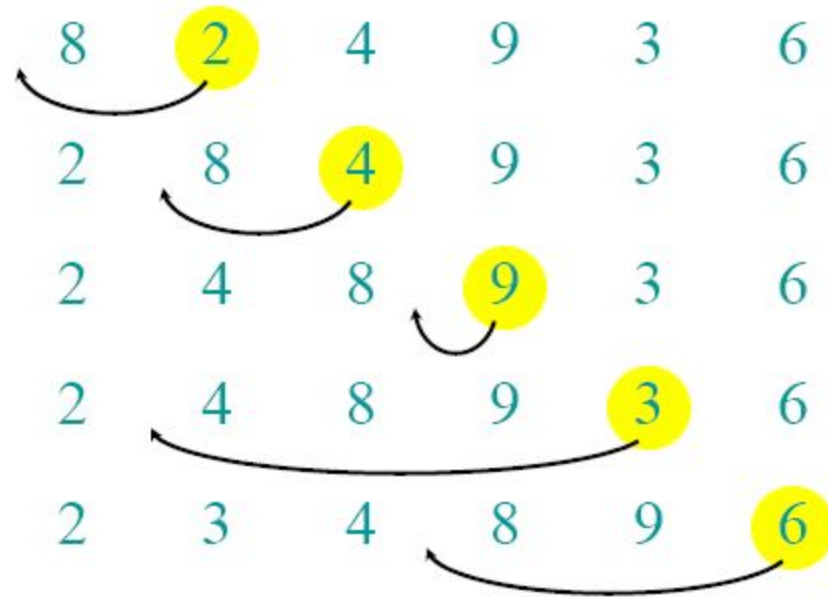
Insertion tracing



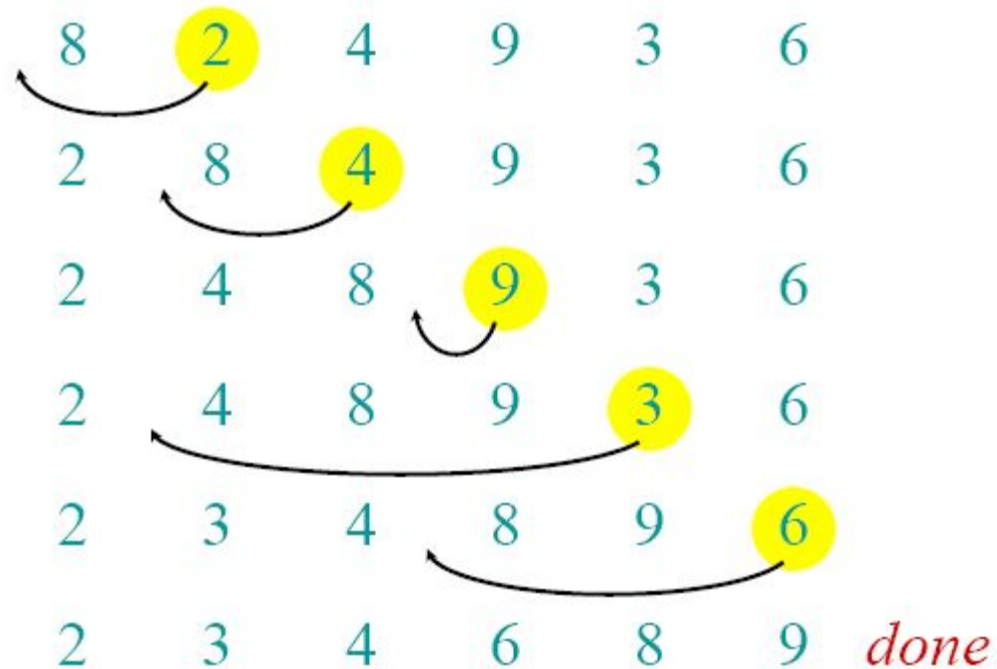
Insertion tracing



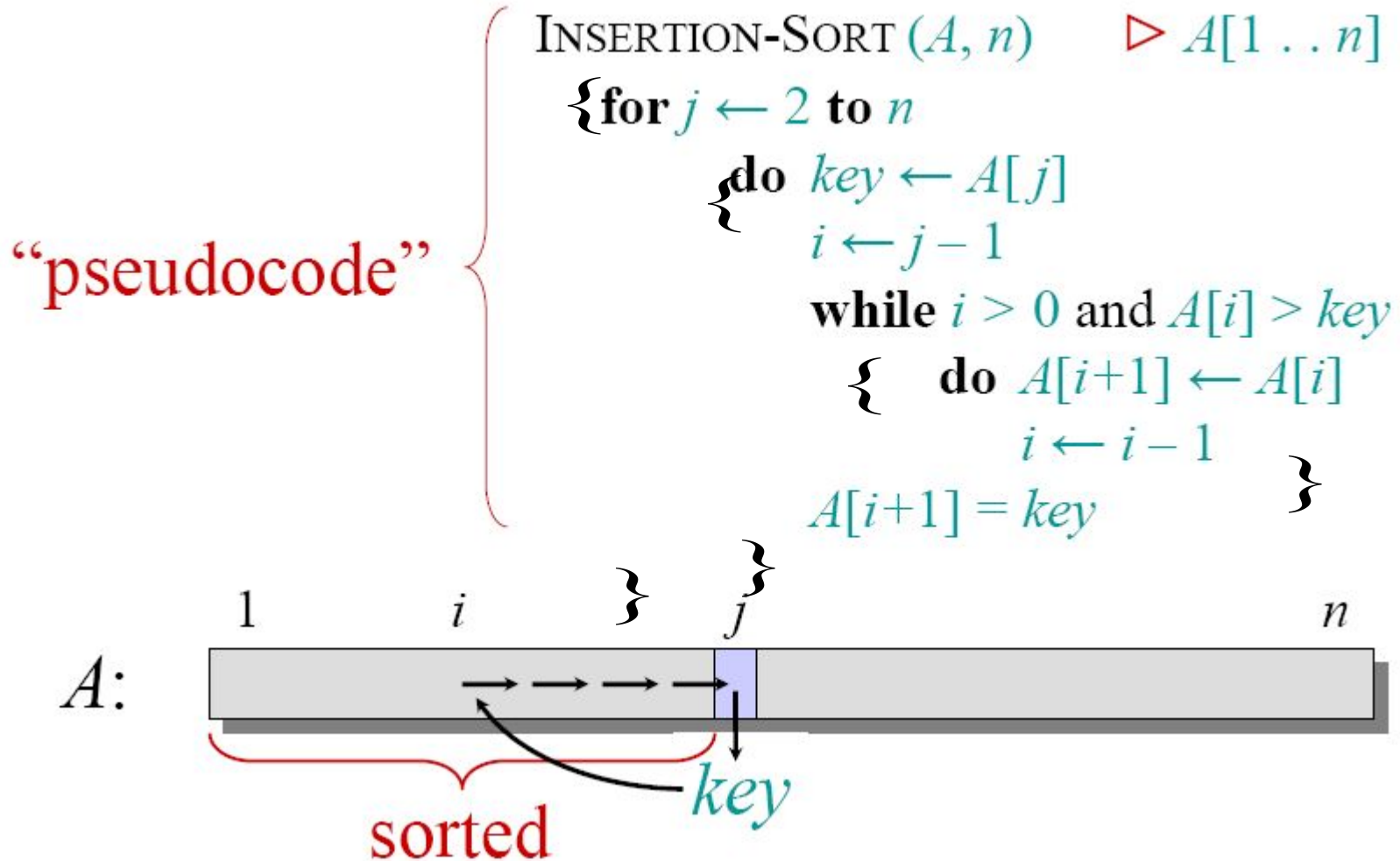
Insertion tracing



Insertion tracing



Insertion Sort



INSERTION-SORT(A)

for $j \leftarrow 2$ **to** n

do $key \leftarrow A[j]$

$\triangleright$ Insert $A[j]$ into the sorted sequence $A[1 \dots j - 1]$.

$i \leftarrow j - 1$

while $i > 0$ and $A[i] > key$

do $A[i + 1] \leftarrow A[i]$

$i \leftarrow i - 1$

$A[i + 1] \leftarrow key$

Details Analysis of Insertion Sort

Execution Steps

```

INSERTION-SORT(A)
{
  for j ← 2 to n
    {
      do key ← A[j]
      // Insert A[j] into the sorted sequence A[1 .. j - 1].
      i ← j - 1
      while( i > 0 and A[i] > key)
        {
          do A[i + 1] ← A[i]
          i ← i - 1
        }
      A[i + 1] ← key
    }
}
    
```

Cost	Time
0	--
0	--
C1	n
C2	n-1
0	--
C3	n-1
C4	$\sum_{j=2}^n (t_j)$
0	--
C5	$\sum_{j=2}^n (t_j - 1)$
C6	$\sum_{j=2}^n (t_j - 1)$
0	--
C7	n-1
0	--
0	--

$$T(n) = c_1 n + c_2(n - 1) + c_3(n - 1) + c_4 \sum_{j=2}^n (t_j) + c_5 \sum_{j=2}^n (t_j - 1) + c_6 \sum_{j=2}^n (t_j - 1) + c_7(n - 1)$$

Best Case: The array is already sorted.

$$T(n) = c_1n + c_2(n - 1) + c_3(n - 1) + c_4 \sum_{j=2}^n (t_j) + c_5 \sum_{j=2}^n (t_j - 1) + c_6 \sum_{j=2}^n (t_j - 1) + c_7(n - 1)$$

INSERTION-SORT(A)

```
{
  for j ← 2 to n
    {
      do key ← A[j]
      // Insert A[j] into the sorted sequence A[1 . . j - 1].
      i ← j - 1
      while( i > 0 and A[i] > key)
        {
          do A[i + 1] ← A[i]
          i ← i - 1
        }
      A[i + 1] ← key
    }
}
```

Always find that $A[i] \leq \text{key}$ upon the first time the while loop test is run (when $i = j - 1$).

- All $t_j = 1$.
- Running time is

$$T(n) = c_1n + c_2(n-1) + c_3(n-1) + c_4(n-2+1) + c_5(n-2+1-1) + c_6(n-2+1-1) + c_7(n-1)$$

You will get

$T(n) = an + b$, linear form

$T(n) = O(n)$

Worst Case: The array is reversed sorted.

$$T(n) = c_1n + c_2(n-1) + c_3(n-1) + c_4 \sum_{j=2}^n (t_j) + c_5 \sum_{j=2}^n (t_j - 1) + c_6 \sum_{j=2}^n (t_j - 1) + c_7(n-1)$$

INSERTION-SORT(A)

```
{
  for j ← 2 to n
    {
      do key ← A[j]
      // Insert A[j] into the sorted sequence A[1 .. j-1].
      i ← j - 1
      while( i > 0 and A[i] > key)
        {
          do A[i + 1] ← A[i]
          i ← i - 1
        }
      A[i + 1] ← key
    }
}
```

We must compare each element $A[j]$ with each element in the entire sorted subarray $A[1 \dots j-1]$, and so $t_j = j$ for $j = 2, 3, \dots, n$.

While loop exit on $i=0$, there is one additional test after $j-1$ tests. i.e $t_j = j$

Now

$$\sum_{j=2}^n (t_j) = \sum_{j=2}^n (j)$$

$$\sum_{j=2}^n (t_j - 1) = \sum_{j=2}^n (j - 1)$$

$$\sum_{j=1}^n (j) \text{ is arithmetic series } = \frac{n(n+1)}{2}$$

$$\sum_{j=2}^n (j) = \sum_{j=1}^n (j) - 1 = \frac{n(n+1)}{2} - 1$$

$$\text{Now, let } K=j-1 \text{ then } \sum_{j=2}^n (j - 1) = \sum_{k=1}^{n-1} (k) = \frac{n(n-1)}{2}$$

$$T(n) = c_1n + c_2(n-1) + c_3(n-1) + c_4\left(\frac{n(n+1)}{2} - 1\right) +$$

$$c_5\left(\frac{n(n-1)}{2}\right) + c_6\left(\frac{n(n-1)}{2}\right) + c_7(n-1)$$

You will get

$$T(n) = an^2 + bn + c, \text{ quadratic form}$$

$$T(n) = O(n^2)$$

Concept of Aggregate Analysis

Aggregate analysis determines the upper bound $T(n)$ on the total cost of a sequence of n operations, then calculates the average cost to be $T(n) / n$.

As a simple example, in a specific implementation of the **dynamic array**, we double the size of the array each time it fills up. Because of this, array reallocation may be required, and in the worst case an insertion may require $\Omega(n)$. However, a *sequence* of n insertions can always be done in $O(n)$ time, because the rest of the insertions are done in constant time, so n insertions can be completed in $O(n)$ time.

The *amortized* time per operation is therefore $O(n) / n = O(1)$.

Concept of Aggregate Analysis

Counter value	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]	Total cost
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	1
2	0	0	0	0	0	0	1	0	3
3	0	0	0	0	0	0	1	1	4
4	0	0	0	0	0	1	0	0	7
5	0	0	0	0	0	1	0	1	8
6	0	0	0	0	0	1	1	0	10
7	0	0	0	0	0	1	1	1	11
8	0	0	0	0	1	0	0	0	15
9	0	0	0	0	1	0	0	1	16
10	0	0	0	0	1	0	1	0	18
11	0	0	0	0	1	0	1	1	19
12	0	0	0	0	1	1	0	0	22
13	0	0	0	0	1	1	0	1	23
14	0	0	0	0	1	1	1	0	25
15	0	0	0	0	1	1	1	1	26
16	0	0	0	1	0	0	0	0	31

Bit Flipping pattern of n bit counter
Can you observe shaded bits pattern ?

Asymptotic Analysis

Best case complexity gives lower bound on the running time of the algorithm for any instance of input(s). This indicates that the algorithm can never have lower running time than best case for particular class of problems.

Worst case complexity gives upper bound on the running time of the algorithm for all the instances of the input(s). This insures that no input can overcome the running time limit posed by worst case complexity.

Average case complexity gives average number of steps required on any instance of the input(s).

In our study we concentrate on worst case complexity only.

Space and Time Complexity

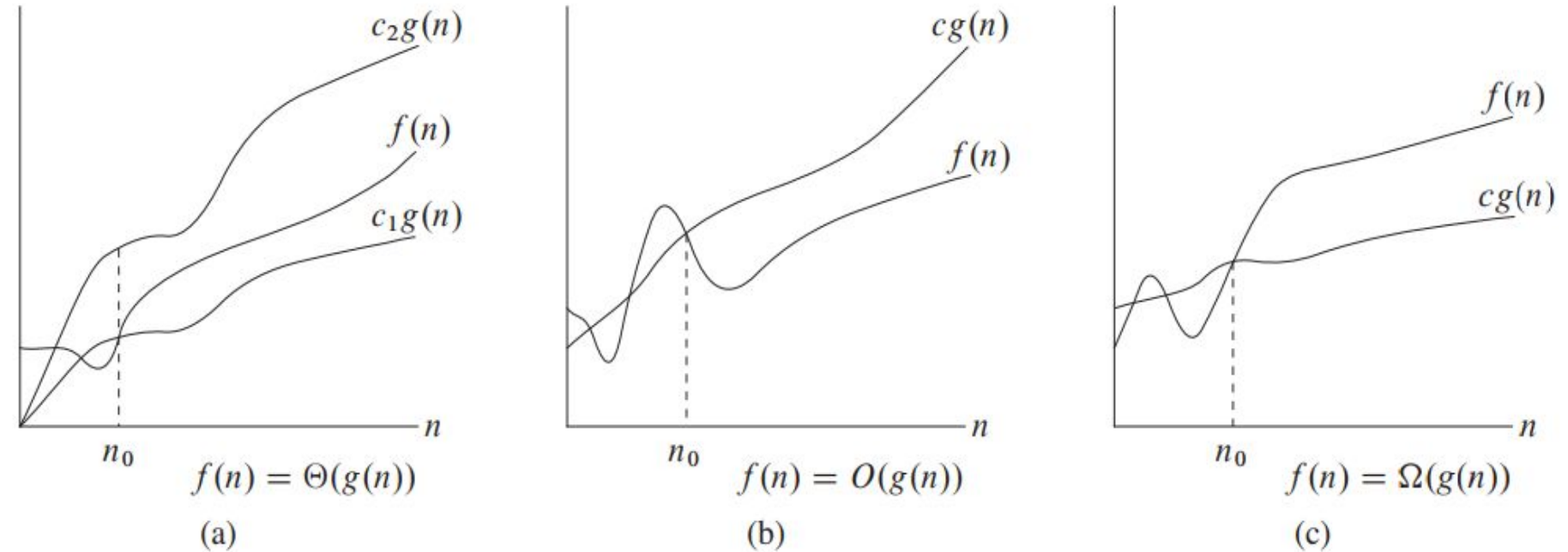
Time Complexity: $T(n)$

Number of steps taken by program

Space Complexity: $S(n)$

Number of variables used in program/size of array used in program

Asymptotic notation



O -notation gives an upper bound for a function to within a constant factor. We write $f(n) = O(g(n))$ if there are positive constants n_0 and c such that at and to the right of n_0 , the value of $f(n)$ always lies on or below $c \cdot g(n)$.

Ω -notation gives a lower bound for a function to within a constant factor. We write $f(n) = \Omega(g(n))$ if there are positive constants n_0 and c such that at and to the right of n_0 , the value of $f(n)$ always lies on or above $c \cdot g(n)$.

Θ -notation bounds a function to within constant factors. We write $f(n) = \Theta(g(n))$ if there exist positive constants n_0 , c_1 , and c_2 such that at and to the right of n_0 , the value of $f(n)$ always lies between $c_1 \cdot g(n)$ and $c_2 \cdot g(n)$ inclusive.

Big-O

$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that}$
 $0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\} .$

If $f(n)=2n^2$ and $g(n)=n^3$

Then show $f(n)=O(g(n))$, with n_0 and c .

Big Omega

$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that}$
 $0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0\} .$

if $f(n) = \sqrt{n}$, and $g(n) = \log n$

Show $f(n) = \Omega(g(n))$

Big -Theta

$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that}$
 $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0\} .^1$

if $f(n) = 1/2n^2 - 3n$, $g(n) = n^2$

Then show $f(n) = \Theta(g(n))$ // check for $c_1 = 1/14, c_2 = 1/2$ and $n_0 = 7$

Use the previous both ideas for these c_1, c_2 and n_0 constants as we discussed in class !

Recursive Algorithms

A **recursive algorithm** is an algorithm which calls itself with "smaller" input values, and which obtains the result for the current input by applying simple operations to the returned value for the smaller input.

More generally if a problem can be solved utilizing solutions to smaller versions of the same problem, and the smaller versions reduce to easily solvable cases, then one can use a recursive algorithm to solve that problem.

Recurrence Relation

- Recursive algorithm can be expressed by recurrence relation.
- Recurrence relation is easy to further interpretation and analysis.
- Recursive programs require more memory and computation compared with iterative algorithms, but they are simpler and for many cases a natural way of thinking about the problem.
- Recurrence relation has base and generalized case.

Recursive Algorithms expression with recurrence relation

1. Recursive algorithm for finding factorial

$$T(n)=1 \quad \text{when } n = 1$$

$$T(n)=T(n-1) + O(1) \quad \text{when } n > 1$$

2. Recursive algorithm for finding Nth Fibonacci number

$$T(1)=1 \quad \text{when } n=1$$

$$T(2)=1 \quad \text{when } n=2$$

$$T(n)=T(n-1) + T(n-2) + O(1) \quad \text{when } n > 2$$

3. Recursive algorithm for binary search

$$T(1)=1 \quad \text{when } n=1$$

$$T(n)=T(n/2) + O(1) \quad \text{when } n > 1$$

Solving Recurrence Relation

1. Iteration method
2. Recursion Tree
3. Substitution
4. Master Method

Iteration method

Method 1: Iteration

$$T(n) = 2T(n/2) + 1 \quad \text{when } n > 1$$

$$T(n) = 1 \quad \text{when } n = 1$$

$$\begin{aligned} T(n) &= 2T(n/2) + 1 \\ &= 2 \{ 2T(n/4) + 1 \} + 1 \\ &= 4T(n/4) + 2 + 1 \\ &= 4 \{ 2T(n/8) + 1 \} + 2 + 1 \\ &= 8T(n/8) + 4 + 2 + 1 \\ &\dots\dots\dots \\ &\dots\dots\dots \\ &= 2^k T(n/2^k) + 2^{k-1} T(n/2^{k-1}) + \dots\dots\dots + 4 + 2 + 1. \end{aligned}$$

For simplicity assume:

$$n = 2^k$$

$$k = \log n$$

$$T(n) = 2^k + 2^{k-1} + \dots\dots\dots + 2^2 + 2^1 + 2^0$$

$$T(n) = (2^{k+1} - 1) / (2 - 1) \quad // \text{ sum of G.S}$$

$$T(n) = 2^{k+1} - 1$$

$$T(n) = 2 \cdot 2^k - 1$$

$$T(n) = 2n - 1$$

$$T(n) = O(n)$$

Solving Recurrence Relation by Recursion Tree

Method 2: Recursion Tree

$$\begin{array}{ll} T(n) = 1 & n = 1 \\ T(n) = T(n/2) + 1 & n > 1 \end{array}$$

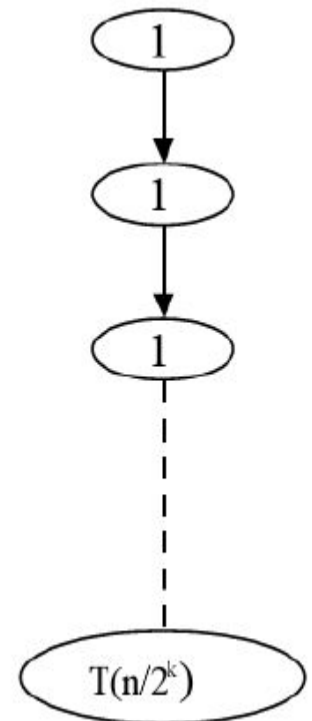
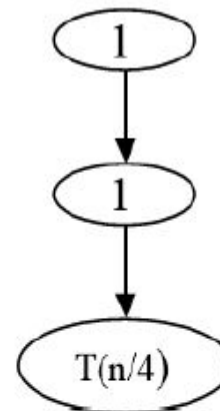
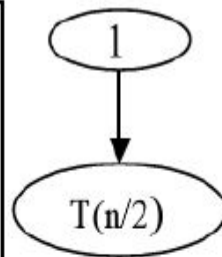
Solution: Drawing recursion tree from given relation and accumulating the cost,

Cost at each level = 1

For simplicity, assume $n = 2^k$

Then $k = \log n$

Total cost = $1 + 1 + 1 + \dots$ upto $\log n$ terms = $O(\log n)$



Solve by recursion tree

$$\begin{array}{ll} T(n) = 1 & n = 1 \\ T(n) = 4T(n/2) + n & n > 1 \end{array}$$

Solve by recursion tree

Method 2: Recursion Tree

$$T(n) = 1 \quad n = 1$$

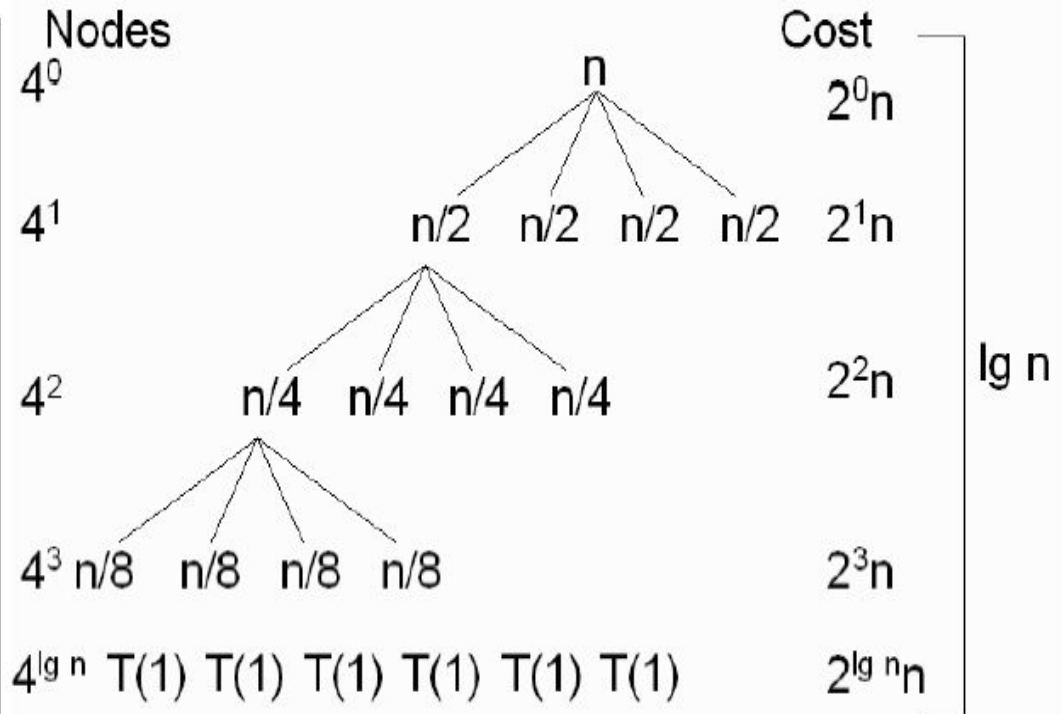
$$T(n) = 4T(n/2) + n \quad n > 1$$

Solution:

→ Bottom level has $n=1$ size problems or $T(1)$

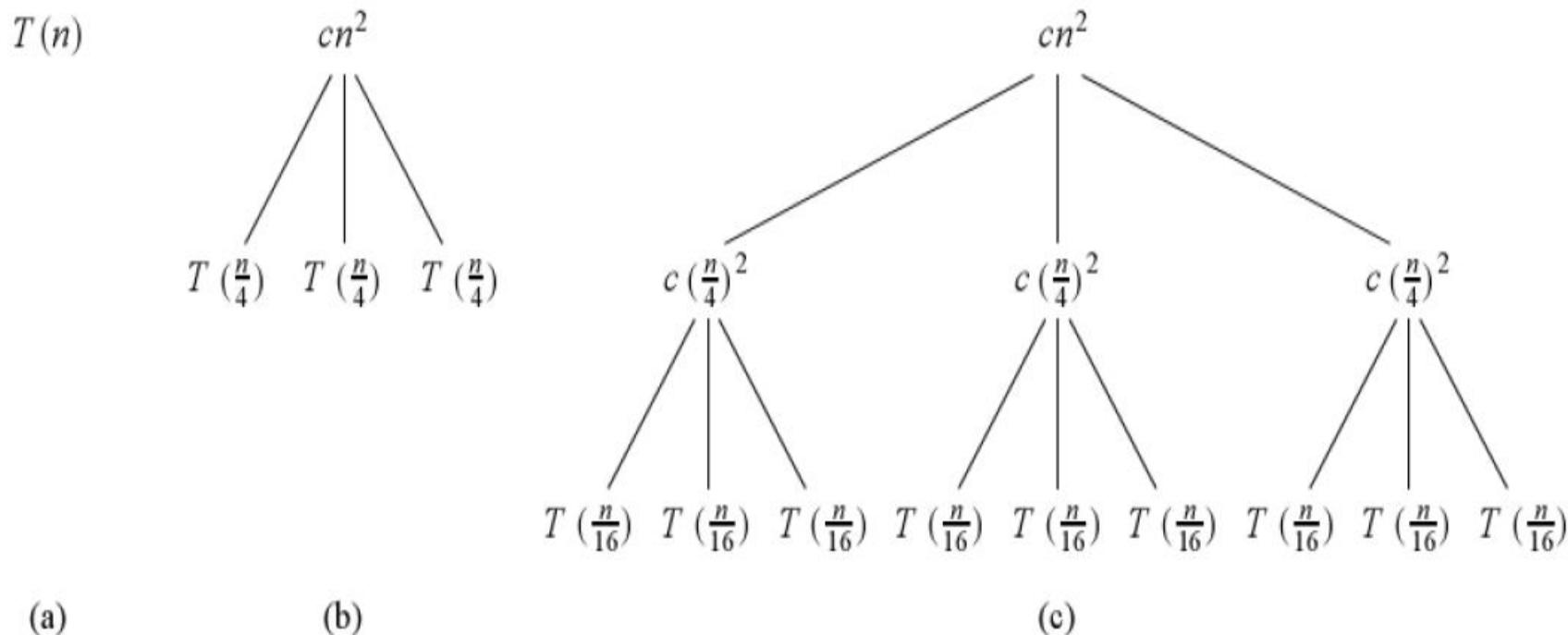
→ Assuming $n = 2^k$, so $k = \log n$.

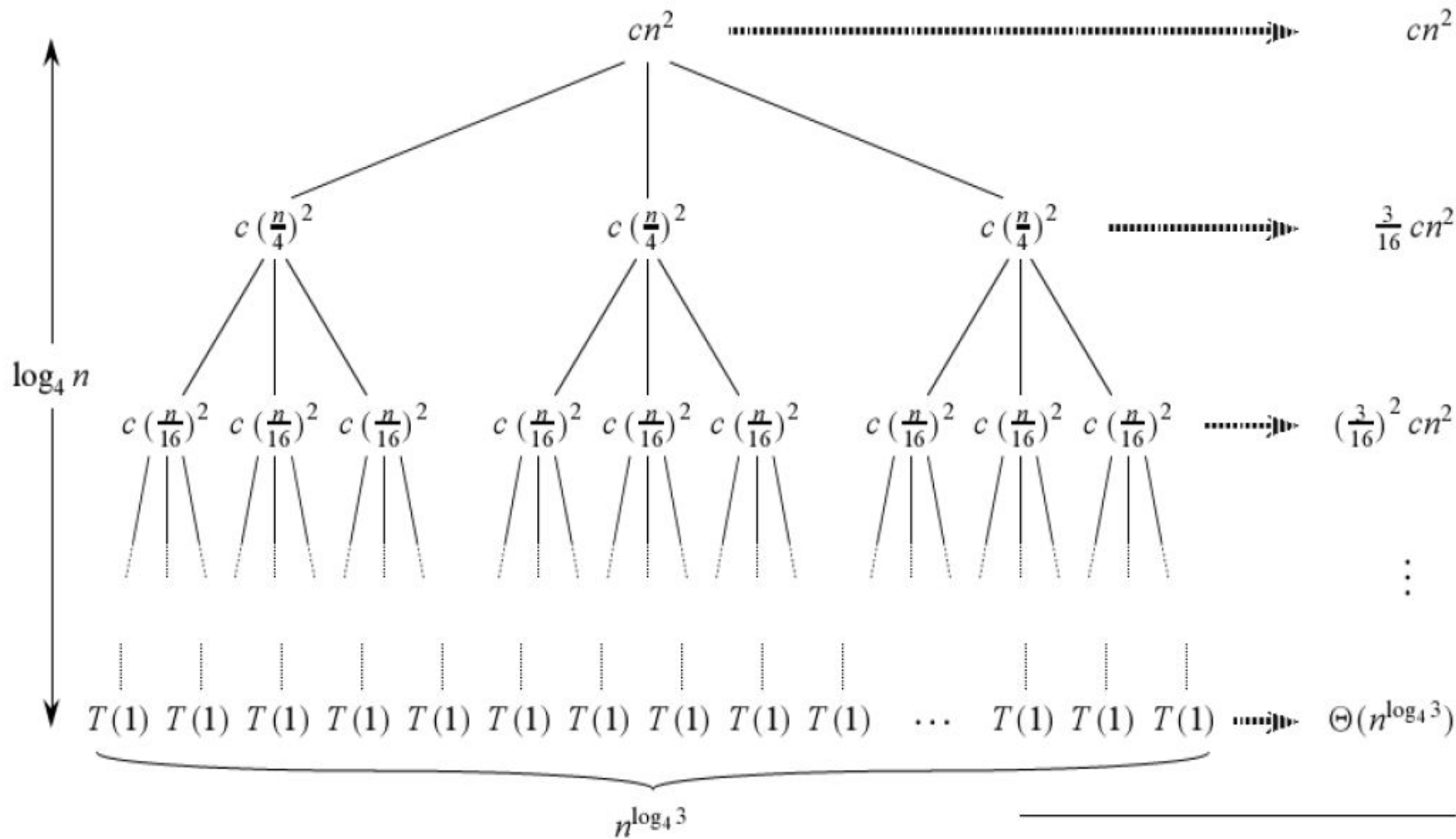
$$\begin{aligned} \rightarrow \text{Total Cost: } & 2^0 n + 2^1 n + 2^2 n + \dots + 2^{k-1} n + 2^k n \\ &= n(1 + 2 + 4 + \dots + 2^{k-1} + 2^k) \\ &= n(2^{k+1} - 1)/(2 - 1) \text{ [Geometric ser.]} \\ &= n(2^{k+1} - 1) \\ &\leq n2^{k+1} \\ &= 2n \cdot 2^k \\ &= 2n \cdot n \\ &= O(n^2) \end{aligned}$$



Example III:

$$T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2)$$





(d)

Total: $O(n^2)$

Total cost

$$\begin{aligned}T(n) &= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\&< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\&= \frac{1}{1 - (3/16)} cn^2 + \Theta(n^{\log_4 3}) \\&= \frac{16}{13} cn^2 + \Theta(n^{\log_4 3}) \\&= O(n^2) .\end{aligned}$$

Case is of decreasing geometric series, so use of

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1-x} .$$

Substitution method

1. Guess the Bound of relation
2. Verify(that guessed bound is right) best guess comes from practice !

Consider the example:

$$T(n) = 1 \quad n=1$$

$$T(n) = 4T(n/2) + n \quad n>1$$

Guess: $T(n) = O(n^3)$.

More specifically:

$T(n) \leq cn^3$, for all large enough n .

For $n=1$:

$$T(n) = 1 \quad \text{Definition}$$

$$1 \leq c \quad \text{Choose large enough } c \text{ for conclusion}$$

Inductive case, $n>1$:

$$T(n) = 4T(n/2) + n \quad \text{Definition.}$$

$$\leq 4c \cdot (n/2)^3 + n \quad \text{Induction}$$

$$= c/2 \cdot n^3 + n \quad \text{Algebra.}$$

While this is $O(n^3)$, we're not done.

Need to show $c/2 \cdot n^3 + n \leq c \cdot n^3$.

Fortunately, the **constant factor** is shrinking, not growing.

$$T(n) \leq c/2 \cdot n^3 + n$$

From before.

$$= cn^3 - (c/2 \cdot n^3 - n)$$

$$\leq cn^3$$

Since $n > 0$, if $c \geq 2$

Proved:

$$T(n) \leq 2n^3 \text{ for } \forall n > 0$$

Thus, $T(n) = O(n^3)$.

Solving Recurrence by Master Method

Master method provides cookbook method for solving recurrences of the form

$$T(n) = aT(n/b) + f(n), \text{ where } a \geq 1, b > 1, \text{ and } f(n) > 0$$

Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined on the nonnegative integers by the recurrence

$$T(n) = aT(n/b) + f(n),$$

where we interpret n/b to mean either $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$. Then $T(n)$ can be bounded asymptotically as follows.

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then $T(n) = \Theta(f(n))$. ■

To use the master method, we simply determine which case (if any) of the master theorem applies and write down the answer.

As a first example, consider

$$T(n) = 9T(n/3) + n.$$

For this recurrence, we have $a = 9$, $b = 3$, $f(n) = n$, and thus we have that $n^{\log_b a} = n^{\log_3 9} = \Theta(n^2)$. Since $f(n) = O(n^{\log_3 9 - \epsilon})$, where $\epsilon = 1$, we can apply case 1 of the master theorem and conclude that the solution is $T(n) = \Theta(n^2)$.

Now consider

$$T(n) = T(2n/3) + 1,$$

in which $a = 1$, $b = 3/2$, $f(n) = 1$, and $n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$. Case 2 applies, since $f(n) = \Theta(n^{\log_b a}) = \Theta(1)$, and thus the solution to the recurrence is $T(n) = \Theta(\lg n)$.

$$T(n) = 3T(n/4) + n \lg n ,$$

we have $a = 3$, $b = 4$, $f(n) = n \lg n$, and $n^{\log_b a} = n^{\log_4 3} = O(n^{0.793})$. Since $f(n) = \Omega(n^{\log_4 3 + \epsilon})$, where $\epsilon \approx 0.2$, case 3 applies if we can show that the regularity condition holds for $f(n)$. For sufficiently large n , $af(n/b) = 3(n/4) \lg(n/4) \leq (3/4)n \lg n = cf(n)$ for $c = 3/4$. Consequently, by case 3, the solution to the recurrence is $T(n) = \Theta(n \lg n)$.

The master method does not apply to the recurrence

$$T(n) = 2T(n/2) + n \lg n ,$$

even though it has the proper form: $a = 2$, $b = 2$, $f(n) = n \lg n$, and $n^{\log_b a} = n$. It might seem that case 3 should apply, since $f(n) = n \lg n$ is asymptotically larger than $n^{\log_b a} = n$. The problem is that it is not *polynomially* larger. The ratio $f(n)/n^{\log_b a} = (n \lg n)/n = \lg n$ is asymptotically less than n^ϵ for any positive constant ϵ . Consequently, the recurrence falls into the gap between case 2 and case 3.